Documentation Algorithmique - Structures de Donnees

Projet Algorithmique Python - Master 1 IdL - 2025/2026

---

1. Dataset brut — avant tokenisation

Champ	Type	Taille	Description
url	str	variable	URL de l'article
texte	str	~2000 car.	Contenu brut de l'article
label	str	1 mot	Categorie : {politique, sport, economie, sante, sciences}
label_id	int	1	Label encode : label_id = LabelEncoder(label) in [0..4]

Taille totale : n = 50 articles, repartis en 5 classes de 10 articles chacune.

Encodage : economie=0, politique=1, sante=2, sciences=3, sport=4

---

2. Tokenisation — transformation du texte en sequences

Champ	Type	Taille	Description
input_ids	list[int]	L = 128	input_ids[i] = identifiant du i-eme token dans le vocabulaire
attention_mask	list[int]	L = 128	attention_mask[i] = 1 si token reel, 0 si padding
labels	int	1	Categorie encodee (label_id)

Pseudo-code :

```
FONCTION tokeniser(texte, max_length=128) :
tokens <- decouper texte en sous-mots (SentencePiece)
input_ids <- [id(token) pour chaque token] + [0] * padding
attention_mask <- [1] * len(tokens) + [0] * padding
RETOURNER (input_ids, attention_mask)
```

Exemple :

```
texte = "Le PSG a gagne."
input_ids = [5, 1234, 456, 78, 910, 6, 0, 0, ...] longueur = 128
attention_mask = [1, 1, 1, 1, 1, 1, 0, 0, ...] 0 = padding
```

Complexite : O(L) ou L = longueur du texte en caracteres.

---

3. Sorties du modele — logits et prediction

Champ	Type	Taille	Description
logits	torch.Tensor	(1, K)	logits[k] = score brut du modele pour la classe k
predictions	int	1	predictions = argmax(logits) = indice de la classe predite
label_predit	str	1 mot	label_predit = classes[predictions]

Pseudo-code :

```
FONCTION predire(texte) :
(input_ids, attention_mask) <- tokeniser(texte)
logits <- modele(input_ids, attention_mask) // vecteur de taille K=5
k* <- argmax(logits) // indice du score max
RETOURNER classes[k*] // label predit
```

Exemple :

```
texte = "Le gouvernement annonce une reforme."
logits = [-1.2, 0.3, 4.5, -0.8, 0.1]
classes = [eco., pol., san., sci., spo.]
```

```
k* = argmax(logits) = 2
prediction = classes[2] = sante
```

Complexite : O(L x H) ou H = 768 (dimension cachee de CamemBERT-base).

---

4. Resume du pipeline

Etape	Entree	Sortie	Complexite
Chargement CSV	articles.csv	DataFrame (n=50, 3 col.)	O(n)
Encodage labels	label (str)	label_id (int) in [0..4]	O(n)
Tokenisation	texte (str)	input_ids, attention_mask (L=128)	O(L)
Prediction modele	input_ids, mask	logits (K=5), argmax	O(L x H)
Evaluation	logits, labels	accuracy, F1-score	O(n)

Tutoriel suivi : https://huggingface.co/docs/transformers/tasks/sequence_classification